

Module 2: Context-Free Languages, Grammars & Modern Parsing

CS 305: Theory of Computation (Weeks 6–8)

Department of Computer Science

Ghana Institute of Management and Public Administration (GIMPA)

Level 300 Undergraduate

Module 2 Roadmap: Weeks 6 to 8

Theoretical Foundations

- **Wk 6:** CFGs, Derivations, Ambiguity & Chomsky Normal Form (CNF)
- **Wk 7:** Pushdown Automata (PDAs) & Stack Memory
- **Wk 8:** Non-CFL Pumping Lemma & Midterm

Research Literature & System Spotlights

- **Wk 6: NeurIPS 2023:** Guided LLM Generation (outlines)
- **Wk 7: PLDI 2011:** ANTLR LL(*) Parser Generators
- **Wk 8: TACL 2020:** Attention Failures on Dyck Languages

Week 6: Context-Free Grammars (CFGs)

Formal Definition (Sipser Def 2.2)

A **Context-Free Grammar (CFG)** is a 4-tuple (V, Σ, R, S) :

- 1 V is a finite set of **variables** (non-terminals).
- 2 Σ is a finite set of **terminals** (disjoint from V).
- 3 R is a finite set of **rules** of the form $A \rightarrow u$, where $A \in V$ and $u \in (V \cup \Sigma)^*$.
- 4 $S \in V$ is the designated **start variable**.

Classic Example: Palindromes and Nested Structures

Grammar $G_1 = (\{S\}, \{0, 1\}, R, S)$ with rules:

$$S \rightarrow 0S0 \mid 1S1 \mid \varepsilon$$

Generates the language of all even-length binary palindromes $L(G_1) = \{ww^R \mid w \in \{0, 1\}^*\}$.

Week 6 Research Spotlight: Guided Generation for Large Language Models

Seminal Reading: NeurIPS Foundation Models Workshop 2023

Willard & Louf, “Efficient Guided Generation for Large Language Models,” **NeurIPS 2023**
(HuggingFace outlines framework).

The Industrial Problem

- LLMs frequently output invalid JSON (missing closing braces, hallucinated keys).
- Breaking API pipelines in production software.

The Theory Solution: CFG Logit Masking

- Convert JSON Schema into an equivalent Pushdown Automaton / CFG parser.
- Mask invalid token logits with $-\infty$ at each decoding step.
- **100% mathematical guarantee of valid JSON!**

Week 7: Pushdown Automata (PDAs)

Formal Definition (Sipser Def 2.13)

A **Nondeterministic Pushdown Automaton (PDA)** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$:

- Γ is the finite **stack alphabet**.
- $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ allows reading input and pushing/popping from the stack.

LIFO Stack

0
0
\$

Main Equivalence Theorem

Language Recognizable by PDA $\iff$ Context-Free Language

Stack memory provides unbounded storage, but only with Last-In-First-Out access.

Week 8: Pumping Lemma for Context-Free Languages

Formal Statement (Sipser Thm 2.34)

If L is context-free, then there exists a pumping length $p \geq 1$ such that any $s \in L$ with $|s| \geq p$ can be written as $s = uvxyz$ satisfying:

- 1 $uv^i xy^i z \in L$ for all $i \geq 0$,
- 2 $|vy| > 0$ (at least one of v or y is non-empty),
- 3 $|vxy| \leq p$.

Classic Non-Context-Free Languages

- $L_1 = \{a^n b^n c^n \mid n \geq 0\}$ (Requires matching 3 quantities $\implies$ single stack fails!).
- $L_2 = \{ww \mid w \in \{0,1\}^*\}$ (Exact duplicate matching $\implies$ non-LIFO!).

Week 8 Research Spotlight: TACL 2020 & Attention Limits on Dyck Languages

Seminal Reading: TACL 2020

Michael Hahn (Stanford Univ.), *"Theoretical Limitations of Self-Attention in Neural Sequence Models,"* Transactions of the Association for Computational Linguistics (TACL 2020).

The Dyck-k Language Problem

- Dyck- k consists of well-nested brackets: $[(\{ \})]$.
- Recognized in $\mathcal{O}(n)$ linear time by a PDA using a stack.

Why Standard Attention Fails

- Self-attention with bounded layers has bounded circuit depth (TC^0).
- Cannot track arbitrary nesting depth beyond its context window without explicit recurrence or scratchpad!

Module 2 Summary: The Boundary of Context-Free Languages

Key Takeaways

- **Context-Free Grammars** and **Pushdown Automata** handle nested, recursive structures (programming languages, JSON, XML).
- A single stack is strictly limited: it cannot verify multi-way constraints like $a^n b^n c^n$ or cross-serial dependencies ww .

Looking Ahead: Module 3 (Turing Machines & Decidability)

What happens if we replace the restricted stack with an **infinite, bidirectional read-write tape**? We unlock the full power of general-purpose computers: **Turing Machines!**